QueryAPIGate DOCUMENTATION · RUN IT

Database connections

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=DATABASE_CONNECTION_CONFIGURATION

Connections live in `queryapigate.db` inside the data folder (`QUERYAPIGATE_HOME`, default: the current directory). `queryapigate init` seeds a starter (inactive templates for every supported database). Every request reads the current state fresh, so edits take effect without a restart, and connections are managed through the `/api/v1/connections` API (or the admin UI, which is built on it).

Shape

One connection, as `POST /api/v1/connections` accepts it - not a literal file anymore, but every field means exactly what it always did:

```json
{
    "name": "reporting",
    "db": "postgres",
    "host": "db.internal",
    "port": 5432,
    "database": "reports",
    "user": "readonly",
    "password": "${REPORTING_PASSWORD}",
    "active": true
}
```

Driver settings beyond these (`sslmode`, `jdbc_url`, `jar`, …) may be sent alongside them or inside an `options` object; `GET /api/v1/connections/{name}` returns them under `options`.

Supported types

DB	DRIVER	NOTES
`mysql`	`mysql-connector-python`	Sessions are opened `READ ONLY` unless writes are enabled
`postgres`	`psycopg2`	Each query runs in a read-only transaction unless writes are enabled
`clickhouse`	`clickhouse-driver` (native protocol, port 9000)	`readonly=1` unless writes are enabled
`sqlite`	`sqlite3`	Opened with `mode=ro` unless writes are enabled
`h2`	`JayDeBeApi` + bundled JDBC jar	Connects to a running H2 TCP server: `jdbc:h2:tcp://<host>[:port]/~/<database>`
`jdbc`	`JayDeBeApi` + your own JDBC jar	Any other JDBC-compliant database (Oracle, SQL Server, DB2, Snowflake, …) - see Generic JDBC connections
`duckdb`	`duckdb`	An embedded analytical database that can also query CSV/JSON/Parquet files directly - see DuckDB connections
`mongo`	`pymongo`	`find()` queries only, never writes - see MongoDB connections
`trino`	`trino` (DB-API)	The Trino distributed SQL engine, federating many catalogs - see Trino connections

`h2`, `jdbc` and `trino` are experimental.

Support matrix

Tier 1 - PostgreSQL, MySQL, SQLite, DuckDB, ClickHouse and, from 0.37, MongoDB - is covered by the compatibility promise and runs against a real server in CI on every push. Types marked `*` are experimental: they work, with the gaps below, and may change in a minor release.

	POSTGRESQL	MYSQL	SQLITE	DUCKDB	CLICKHOUSE	H2 *	JDBC *	MONGODB	TRINO *
Saved queries and ad-hoc runs	yes	yes	yes	yes	yes	yes	yes	find() only	yes
Read-only by default (SQL check)	yes	yes	yes	yes	yes	yes	yes	always (find() only)	yes
Read-only enforced by the database too	yes	yes	yes	no	yes	no	no	n/a	no
Writes, when allowed	yes	yes	yes	yes	yes	yes	yes	no	yes
Query time limit	yes	SELECT only	yes	yes	yes	yes	no	yes	yes
`allowed_tables` (refused where not supported)	yes	yes	yes	yes	yes	no	no	collections	yes
Schema browser, MCP `list_tables`	yes	yes	yes	yes	yes	yes	no	yes	yes
Primary and foreign keys in the schema	yes	yes	yes	yes	no	no	no	n/a	no
Listing and switching databases	yes	yes	n/a	n/a	yes	no	no	yes	yes
Table DDL	no	yes	yes	no	yes	no	no	n/a	no
Descriptions from database comments	yes	yes	n/a	yes	yes	no	no	n/a	no
Streaming exports	yes	yes	yes	yes	yes	yes	yes	yes	yes
Files: Parquet, CSV, JSON, local or on S3/GCS/R2/HTTP	no	no	no	yes	no	no	no	no	no
Response caching (`cache_ttl`)	yes	yes	yes	yes	yes	yes	yes	no	yes
MCP `execute_sql`	yes	yes	yes	yes	yes	yes	yes	no	yes
Tables-and-joins diagram	yes	yes	yes	yes	yes	no	no	n/a	yes
Tested against a real server in CI	yes	yes	yes	yes	yes	yes	yes	yes	yes

`n/a` is a feature the database has no use for (a SQLite file has no other database to switch to; MongoDB has no SQL). MongoDB's read-only default is `always`: it runs `find()` and nothing else, so there is no write to refuse. Where `allowed_tables` says `no`, a key restricted to tables is refused on that connection rather than trusted, and `Read-only enforced by the database too: no` means the read-only default rests on QueryAPIGate's own SQL check.

An experimental type becomes tier 1 when:

1. It runs against a real server in the CI integration job (tests/test_integration.py) - reads, and writes where it takes them;
2. The read-only default, the query time limit and `allowed_tables` all work on it - each a test, not a claim;
3. Its gaps in this table are closed, or stated as limits of the database itself;
4. Its entry leaves `queryapigate/experimental.py`, and this table and the changelog say so.

The SQL guard (single-statement / read-only check, see API.md) reads string literals using the quoting rules the connection's `db` type actually uses: `mysql` and `clickhouse` honour a backslash escape inside quoted strings by default, the others do not, and using the wrong rule for a value can misjudge where a statement ends.

Query time limit

`QUERYAPIGATE_QUERY_TIMEOUT` (default 30 seconds) is enforced by each database itself, so the statement is genuinely cancelled and its resources released rather than merely abandoned:

DB	MECHANISM	NOTES
`mysql`	`max_execution_time` (MariaDB: `max_statement_time`)	Applies to `SELECT`. MySQL cuts `SLEEP()` and `BENCHMARK()` short but returns normally instead of raising an error.
`postgres`	`statement_timeout`	
`clickhouse`	`max_execution_time`	Whole seconds, checked as data blocks are processed, so cancellation can lag slightly.
`sqlite`	progress handler	Checked every 10 000 VM instructions.
`h2`	`SET QUERY_TIMEOUT`	
`jdbc`	*not enforced*	No portable way to cancel a statement across arbitrary JDBC drivers - see Generic JDBC connections
`duckdb`	`Connection.interrupt()` on a background timer	
`trino`	`cursor.cancel()` on a background timer, plus a `query_max_run_time` session backstop	Cancels the running query on the Trino cluster.

Connection pooling

MySQL, PostgreSQL, ClickHouse, H2, generic `jdbc` and `duckdb` connections are kept open and reused between requests (SQLite is a local file and is opened per request). Tune it with `QUERYAPIGATE_POOL_SIZE` (idle connections kept per distinct connection setting, default 8, `0` disables pooling) and `QUERYAPIGATE_POOL_IDLE_TIMEOUT` (seconds, default 300).

- **Clean hand-over.** A connection's transaction is ended before it is reused, so a request never sees a stale snapshot, and query limits are applied per request (or per transaction) so they never leak to the next user.
- **Errors.** A connection used by a failed or timed-out request is closed rather than reused.
- **Health checks.** A connection that sat idle for more than a few seconds is checked before reuse; a dead one is replaced transparently.

- **Changes take effect.** Creating, changing or deleting a connection closes all idle pooled connections immediately. If you edit `queryapigate.db` directly (e.g. via `sqlite3`), old connections are dropped as they reach the idle timeout.
- **Sizing.** The pool bounds *idle* connections, not concurrent ones. Each server process has its own pool, so the most idle connections your database sees is roughly `QUERYAPIGATE_POOL_SIZE` x distinct connections x worker processes; keep that below the database's `max_connections`.
- **Streaming exports hold their connection for the whole download, not just the query.** `?stream=true` (see Streaming exports) checks a connection out of the pool exactly like any other request, but does not release it back until the client has received the *entire* result - which, for a slow client or a very large export, can be a lot longer than a normal request. A few large concurrent exports can make a small `QUERYAPIGATE_POOL_SIZE` feel undersized for everything else running at the same time; size accordingly, or set `QUERYAPIGATE_POOL_SIZE=0` if that trade-off is not acceptable for your deployment (every request, including streamed ones, then opens and closes its own connection).
- **Streaming's own memory footprint varies by dialect.** MySQL (an unbuffered cursor), PostgreSQL (a named, server-side cursor) and ClickHouse (`execute_iter`) stream from the database itself without the driver buffering the whole result client-side first - verified against a real server: streaming 1 million rows kept this project's own process memory flat throughout, against several hundred MB to fetch the same result the ordinary way. SQLite, H2, the generic `jdbc` type and DuckDB still bound *this project's own* memory to one batch (1000 rows) at a time regardless of result size, but the underlying engine or driver may still hold more than that internally - H2/`jdbc` because jaydebeapi exposes no way to set the JDBC `ResultSet`'s fetch size, DuckDB because its engine computes the whole result during `execute()` before the first row is even fetched (see the `_DuckDB` / `_H2` docstrings in `runners.py` for what was actually measured, not assumed).

Properties

PROPERTY	REQUIRED	DESCRIPTION
`db`	yes	One of the types above
`active`	yes	Only active connections can be used (otherwise 403)
`database`	yes	Database name, or the file path for SQLite/DuckDB (relative paths resolve against the data folder)
`host`	network databases	Server host
`port`	no	Overrides the driver's default port
`user`, `password`	usually	Credentials
`jar`, `driver_class`, `jdbc_url`	`db: "jdbc"` only	See Generic JDBC connections
`options.schema`	no	Trino: the default schema. PostgreSQL: the schema an unqualified `allowed_tables` name means, `public` by default (a table-restricted key's calls run with `search_path` set to it)

Experimental: h2, jdbc and trino

> **Experimental** - may change in any minor release, always noted in the changelog (what that means).

These types work, with the gaps the support matrix shows: on H2 and JDBC, `allowed_tables` refuses queries rather than guess (fail closed); all three rest their read-only default on the SQL check alone; and JDBC has no query time limit and no schema browser. **Trino** (ADR 0026) is fully governed - masking, the schema browser, the query time limit and catalog/schema-qualified `allowed_tables` all work. A read-only, catalog/schema-scoped Trino user is still recommended as an outer boundary; the federation boundary has a security review (the threat model's "Trino" section), and what remains before it graduates is a release of unchanged shape and real use. H2, JDBC and Trino are all tested against a real server in CI. A server with an experimental connection configured logs a warning at startup saying so. The matrix also lists what each needs to become tier 1. (MongoDB was experimental too, until 0.37 - below.)

MongoDB connections

> **Stable** since 0.37, under the compatibility promise (what that means): the `mongo` saved-query fields (`mongo_collection`, `mongo_filter`, `mongo_projection`, `mongo_sort`), `/execute_mongo`, and what each streamed format writes. Aggregation pipelines, writes, response caching, column masking and incremental exports may still be added. Limits: `find()` only - no writes, no response caching, no MCP `execute_sql`, no column masking (a caller with mask rules is refused); a streamed CSV or TSV takes its columns from the projection or the first 1,000 documents; and `allowed_tables` matches collection names case-insensitively, so a grant of `orders` also reads a collection named `Orders`.

`find()` queries - filter, projection, sort, bound `:name` parameters - ad hoc through `POST /execute_mongo`, or saved and called at `/q/<name>` like any SQL query (Connect to MongoDB). Read-only by construction: nothing but `find()` is ever sent. `$where`, `$function` and `$accumulator` - server-side JavaScript - are refused in the filter, the projection and the sort, and a parameter's value is a value, never a document of operators. `allowed_tables` is the collections a key may read, checked before the server is asked anything; the query time limit is the cursor's `maxTimeMS`.

Streaming (`?stream=true` on `/q/<name>` and `/execute_mongo`, `queryapigate export`, saved exports - ADR 0022). Documents don't share a schema, so each format decides:

FORMAT	WHAT IT WRITES
NDJSON	Each document whole: nested documents and arrays stay nested, no field is dropped
Parquet	Every field of every document as a column, in first-seen order; a field holding a number in one document and text in another is a text column; nested values are JSON text
CSV, TSV (streamed)	The columns are decided before the first row: an inclusion projection's fields in its order (`_id` first unless `"_id": 0`), or else the fields of the first 1,000 documents. A field first seen later is left out, and the run's history counts them (`fields_dropped`); nested values are JSON text
Any format, to a destination	Staged before it's written, so every field of every document is a column, as for Parquet - nested values JSON text, NDJSON included

For a CSV whose columns don't depend on the data, name them in the projection. For an export that keeps every field and its nesting, use NDJSON. Incremental exports (`incremental`) of a `mongo` query are refused, for now; a full export works.

Generic JDBC connections

`db: "jdbc"` reaches any database with a JDBC driver - Oracle, SQL Server, DB2, Snowflake and others not covered by a dedicated driver above - by generalising the same embedded-JVM approach `h2` already uses. It needs three fields instead of `host` / `port` / `database`, since JDBC URL formats vary too much between vendors to build one generically:

```json
{
    "oracle-reporting": {
        "db": "jdbc",
        "jar": "/opt/jdbc/ojdbc11.jar",
        "driver_class": "oracle.jdbc.OracleDriver",
        "jdbc_url": "jdbc:oracle:thin:@//db.internal:1521/ORCLPDB1",
        "user": "readonly",
        "password": "${ORACLE_PASSWORD}",
        "active": true
    }
}
```

- `jar` - path to the vendor's JDBC driver `.jar` (not bundled - only H2's is). Needs a JVM, so either the `-h2` Docker image variant or your own JVM installation, same as `h2`.
- `driver_class` - the driver's fully-qualified Java class name (from its documentation, e.g. `oracle.jdbc.OracleDriver`, `com.microsoft.sqlserver.jdbc.SQLServerDriver`, `com.ibm.db2.jcc.DB2Driver`, `net.snowflake.client.jdbc.SnowflakeDriver`).
- `jdbc_url` - the complete JDBC connection URL, exactly as that vendor's driver expects it.

Three consequences of reusing one embedded JVM, not specific to any one connection:

- **The JVM's classpath is fixed the moment it starts** (from whichever `h2` or `jdbc` connection is used first), and cannot be changed afterwards. Every jar from every `jdbc` connection configured *at that moment* is included automatically, so the common case - configure your connections, then start using them - works with no extra steps. Adding a **new** `jdbc` connection whose jar isn't already on the classpath needs the server **restarted** before that connection can be used.
- **No query time limit is enforced** (see the table above) - `QUERYAPIGATE_QUERY_TIMEOUT` does not cancel a slow query on a `jdbc` connection.
- **Schema introspection is not available**: `GET /connections/<name>/schema` answers 400 for a `jdbc` connection - vendor system-catalogue queries differ too much to generalise safely yet.

Like `h2`, a `jdbc` connection's read-only mode rests on the SQL guard alone (`Connection.setReadOnly()` is advisory in the JDBC specification, not something every driver is required to enforce).

DuckDB connections

`db: "duckdb"` is an embedded analytical database - its own storage, its own `.duckdb` file, no server process to run - that also reads files directly: Parquet, CSV and JSON, on local disk, object storage or the web. One connection type, two uses:

```json
{"name": "analytics", "db": "duckdb", "database": "/data/analytics.duckdb", "active": true}
```

As a general embedded database, it behaves like `sqlite`: point `database` at an existing file (QueryAPIGate never creates one, so a mistyped path fails instead of opening an empty database - create it first, e.g. `python -c "import duckdb; duckdb.connect('analytics.duckdb')"`), then `CREATE TABLE` / `INSERT` / `SELECT` against it as usual. Its SQL dialect is close to PostgreSQL, so it uses the same quote-doubling string-literal rules as `postgres` / `sqlite` / `h2` (no backslash escaping). Or set `database` to `":memory:"` for a connection that holds no tables of its own - only views over files (below).

For files, list what the connection may read in `allowed_paths`, then read them straight from SQL with DuckDB's own table functions - no import step, column types inferred:

```json
{"name": "files", "db": "duckdb", "database": ":memory:", "active": true,
 "allowed_paths": ["/data/sales/", "/data/reference/countries.csv"]}
```

```sql
SELECT customer_id, sum(amount) FROM read_parquet('/data/sales/*.parquet') GROUP BY customer_id
SELECT * FROM read_csv('/data/reference/countries.csv') WHERE region = :region
```

Nothing outside `allowed_paths` can be read - not another file, folder, URL or database, by any route (`read_csv`, `FROM 'file.parquet'`, `glob`, `ATTACH`, `COPY`), and no extension can be installed or loaded. DuckDB itself enforces this: each connection is locked as it opens (`enable_external_access = false`, the allowed paths, `lock_configuration`), so a query can't change it back. A refused path answers `403 path_not_allowed`. **A connection with no `allowed_paths` reads no files at all** - only its own database.

- An entry ending in `/` allows that folder and everything under it; anything else allows exactly that file.
- Local entries must be **absolute**, and SQL must use the same absolute path: DuckDB checks a path as written, so a relative one never matches.
- `..` and wildcards aren't accepted in entries; a `..` in a query's path is resolved before the check, so it can't climb out.

Views give files a name - for the schema browser, for saved queries, and for `allowed_tables`:

```json
{"views": {"sales": "SELECT * FROM read_parquet('/data/sales/*.parquet')"}}
```

Each is created as a temporary view on every new connection. A key restricted with `allowed_tables: ["sales"]` can query the view but not call `read_parquet` itself: for a table-restricted key, table functions that read data are refused (`table_not_allowed`).

Automatic views - `"auto_views": true` - name the files for you: each time a connection opens, every allowed folder (or bucket prefix) is listed, through the locked connection itself, and each file and subfolder becomes a view:

UNDER `/DATA/LAKE/`	VIEW	READS
`orders.parquet`	`orders`	that file
`events/day=1/part.parquet`, `events/day=2/...`	`events`	`events/**/*.parquet`, with `day` as a column (hive-style `key=value/` folders)
`mixed/a.csv` and `mixed/b.json`	`mixed_csv`, `mixed_json`	one view per kind of file in the folder
`2026 Sales.csv`	`v_2026_sales`	names are lower-cased, other characters become `_`, a leading digit gets `v_`

An allowed **file** (a web address, say) becomes a view named after it. Recognised: `.parquet`, `.csv`, `.tsv`, `.json`, `.ndjson`, `.jsonl` (and `.gz`/`.zst`-compressed CSV and JSON); other files are ignored. A view in `views` with the same name wins. A file that can't be read loses its view, with a warning in the log (`auto_views: skipped broken: ...`), rather than failing the connection. Listing happens on each new pooled connection - cheap for a folder, a list request for a bucket; at most 10,000 files are looked at per allowed folder. In the Console, **Files (Parquet, CSV, JSON)** as the type creates exactly this: `database: ":memory:"`, `auto_views: true`.

Two things carried over from `h2`/`jdbc`, both for the same underlying reason (DuckDB refuses to open a second connection to a file with a different read-only setting than a connection already open on it, which pooling read-only and read-write connections separately would trip constantly):

- Every pooled connection is opened read-write regardless of the caller's read-only mode - **the read-only guarantee rests on the SQL guard alone**, same as `h2`/`jdbc`.
- Unlike `h2`/`jdbc`, the query time limit **is** enforced (`Connection.interrupt()`, see the table above) - this is a DuckDB Python client capability the JDBC-based drivers don't have access to.

Trino connections

> Experimental (ADR 0026). `trino` turns the Trino distributed SQL engine - which federates many catalogs (Hive, Iceberg, Delta, Kafka, object stores, other databases) behind one endpoint - into a governed source. Install with `pip install "queryapigate[trino]"`.

```json
{
  "name": "warehouse", "db": "trino",
  "host": "trino.internal", "port": 8080, "user": "queryapigate",
  "database": "hive",
  "options": { "schema": "analytics", "http_scheme": "https" }
}
```

- **`database` is the Trino *catalog*.** The optional `options.schema` sets the default schema; `options.http_scheme` is `http` or `https`. A `password` is sent as HTTP basic auth over `https`. "Switch database" in the Console lists catalogs (`SHOW CATALOGS`) and switches between them.
- **The schema browser and MCP `list_tables`** read the catalog's `information_schema`, scoped to the connection's default schema (`current_schema`). Primary/foreign keys and table DDL are not shown (Trino has no enforced constraints, and three-part identifiers need more handling) - as for ClickHouse.

- **Governance is the same as any SQL connection:** column masking, the tables-and-joins diagram and `allowed_tables` all work through sqlglot's `trino` dialect; the query time limit cancels the running query on the cluster.
- **`allowed_tables` is catalog/schema-qualified** (ADR 0026): a grant of `orders` means the connection's default catalog and schema's `orders` only - never `orders` in another catalog or schema. Grant elsewhere by writing `schema.table` or `catalog.schema.table`. A read-only, catalog/schema-scoped Trino user is still recommended as an outer boundary; QueryAPIGate's grants narrow within it.
- **Read-only** rests on the SQL guard alone (Trino has no read-only session mode), same as `h2` / `jdbc` / `duckdb`. **Agent writes are not supported** on Trino in this release.

Files on S3, GCS, R2 and the web

> **Stable** since 0.20, under the compatibility promise (what that means). New storage types and connection fields may still be added. Limits: GCS and R2 are read through their S3-compatible APIs (tested against an S3-compatible store, not GCS or R2 themselves), and credentials are given - AWS instance roles aren't used.

`allowed_paths` also takes object-storage prefixes and web addresses, read through DuckDB's `httpfs` extension (in the Docker image; elsewhere it's downloaded on first use, or install it ahead with `python -c "import duckdb; duckdb.connect().execute('INSTALL httpfs')"`):

```json
{"name": "lake", "db": "duckdb", "database": ":memory:", "active": true,
 "allowed_paths": ["s3://sales/2026/", "https://data.example.com/prices.parquet"],
 "user": "AKIA...", "password": "${LAKE_SECRET_KEY}", "region": "eu-west-1",
 "views": {"orders": "SELECT * FROM read_parquet('s3://sales/2026/*.parquet')"}}
```

FIELD	MEANING
`allowed_paths`	`s3://`, `gs://` (or `gcs://`) and `r2://` prefixes ending in `/`; `http(s)://` **files** - a web address must name one file, because a web server resolves `..` in a URL and a prefix couldn't stop that
`user`, `password`	the access key id and secret - stored like any connection password: masked, encrypted with `QUERYAPIGATE_SECRET_KEY`, or a `${VAR}` reference. Leave both out for public files
`storage`	`s3` (default, also any S3-compatible service), `gcs` (HMAC keys) or `r2`
`region`, `endpoint`, `url_style`, `use_ssl`	as DuckDB's S3 settings: `endpoint` and `url_style: "path"` for an S3-compatible service such as MinIO; `account_id` for R2

A bucket prefix is a second lock, not the first. DuckDB refuses anything outside the listed prefixes, but an S3-compatible server might itself resolve `..` in an object key, so give the connection credentials that can read only what it should - a bucket policy or IAM role scoped to those prefixes. Tested against an S3-compatible server (SeaweedFS) and a plain web server; Google Cloud Storage and R2 use the same mechanism but weren't tested here.

Any caller with the connection in its `connections` grant can read every listed path with its own SQL; the key ID and endpoint are visible to it through DuckDB's `duckdb_secrets()` (the secret itself is always redacted). For partners, publish saved queries or views and grant those instead.

Keeping secrets out of the file

Any string value may contain `${VAR}` references, replaced with the environment variable's value when the connection is used (a missing variable is reported as an error naming it). Set `QUERYAPIGATE_SECRET_ENV_VARS` to the variables your connections use - names, or prefixes ending in `*`: `REPORTING_PASSWORD,ORACLE_*` - and no other can be referenced. Without it, any can but the server's own settings and the credentials it may run with (below), and the server warns at startup while references are in use. `GET /api/v1/connections/{name}` masks plain-text passwords as `********` and shows `${VAR}` references as written. A secret kept in a file - the CSI driver's, Docker's, Vault Agent's - is `${file:/run/secrets/name}`, read inside the folders `QUERYAPIGATE_SECRET_FILE_DIRS` allows (Secrets from files and secrets managers).

Best practice

- Use a database account with only the privileges the API needs. The read-only guard is defence in depth.
- Keep `queryapigate.db` out of version control (the repository's `.gitignore` already does).
- Use `"active": false` to disable a connection without deleting it.

READ NEXT
API reference

QUESTIONS OR PROBLEMS
Discord · GitHub issues

LATEST VERSION OF THIS PAGE
anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=DATABASE_CONNECTION_CONFIGURATION

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit
@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate